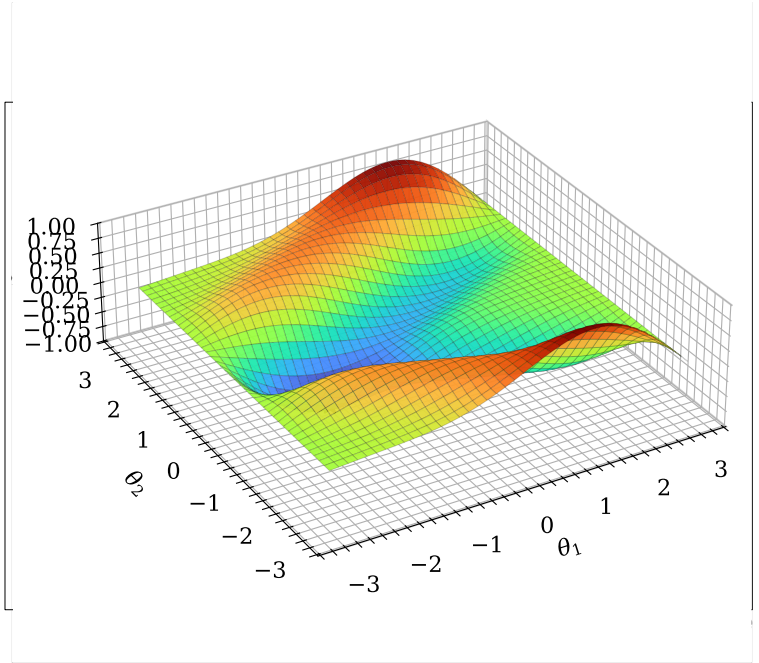
BASICS OF FIM · WI001287

Statistical Learning

5: Neural networks



Univ.-Prof. Dr.-Ing. habil. Niklas Kühl
Lars Böcking

Chair for Information Systems and
Human-centric Artificial Intelligence
University of Bayreuth
TUM School of Management

A straight line underfits; disaster patterns are nonlinear.

On the disaster data (Earthquake vs. Flood by longitude/latitude): linear draws a straight boundary, logistic a curved one, a tree blocky regions.

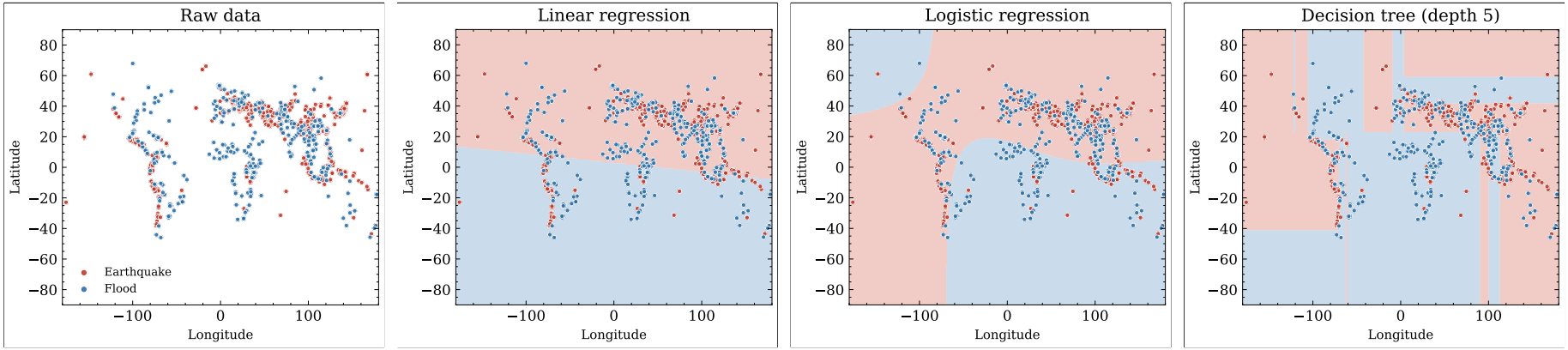


Figure 5.1: motivation

We want to learn a smooth, nonlinear decision boundary

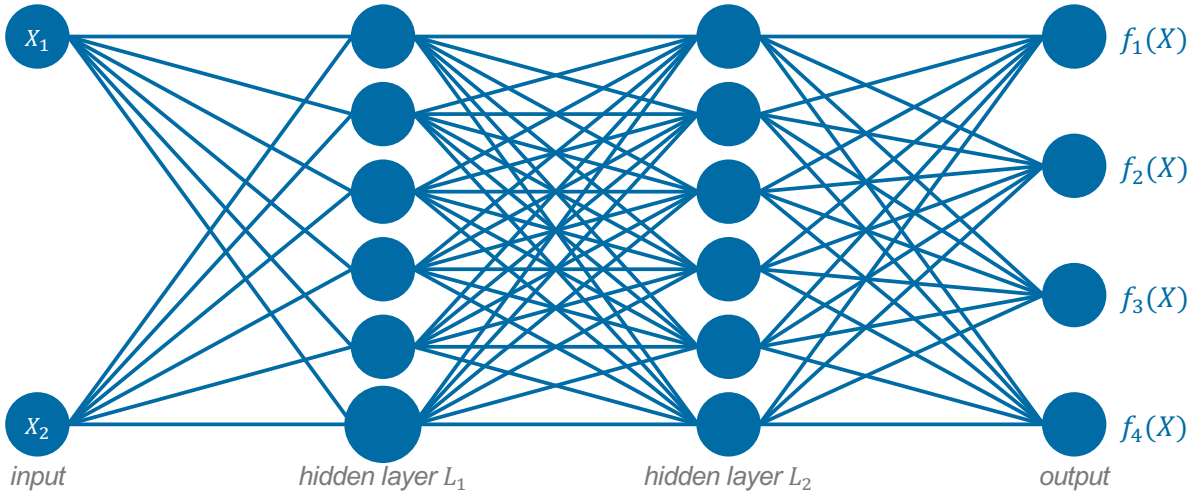
Agenda

- Motivation for neural networks
- Network architecture
- Loss and backpropagation
- Optimization

A neural network stacks an input, hidden, and output layer.

Inputs $X = (X_1, \ldots, X_p)$ feed the *brain*.
The output layer produces $f(X)$, where the index indicates the different target classes in case of classification.

A neural network stacks an input, hidden, and output layer.

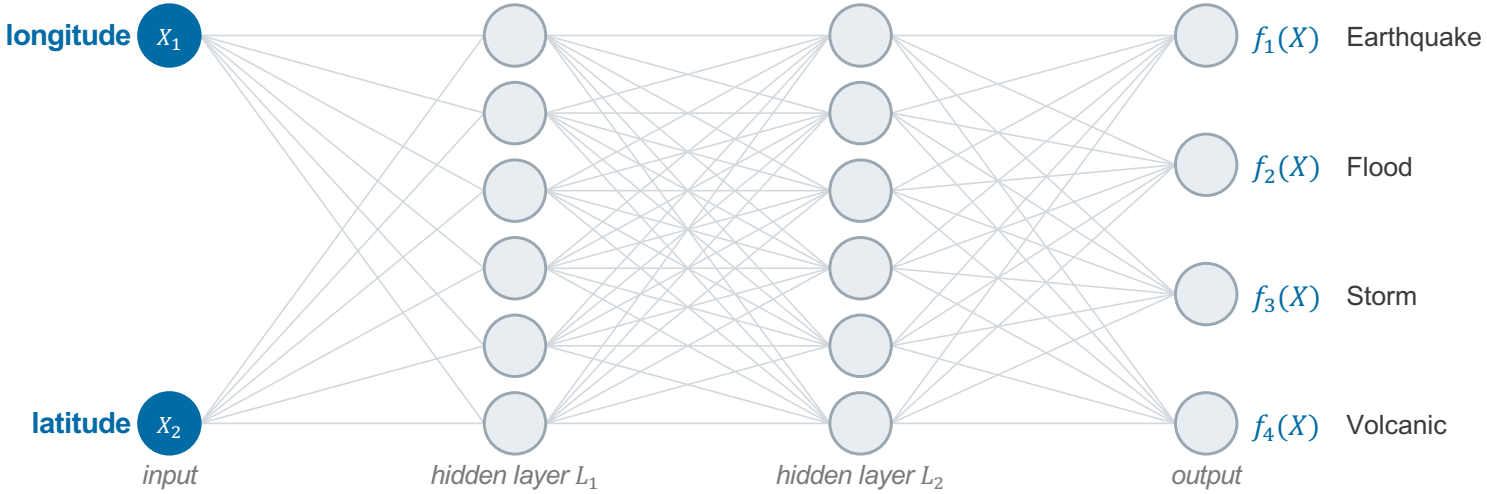


Inputs $X = (X_1, \dots, X_p)$ feed the hidden layers.
The output layer produces $f(X)$, where the index indicates the different target classes in case of classification.

Agenda

- Motivation for neural networks
- Network architecture
- Loss and backpropagation
- Optimization

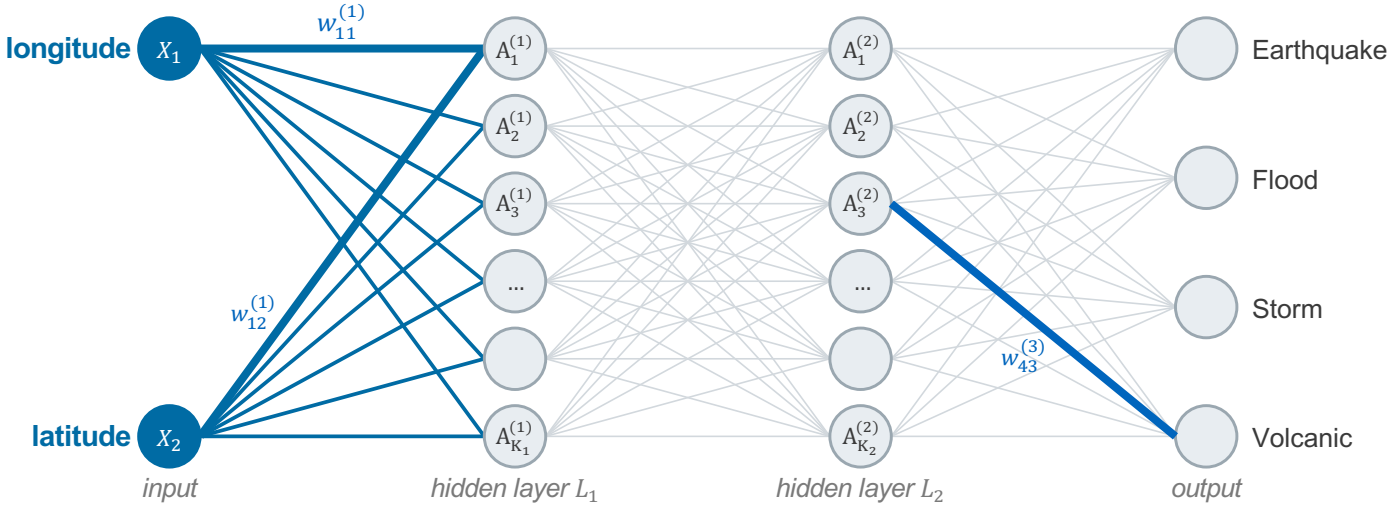
The input layer holds the p features we feed the network.



$$X = (X_1, X_2)$$

here X_1 = longitude, X_2 = latitude and $f_1(X)$= Earthquake etc.

Weights on the edges scale every signal that flows forward.

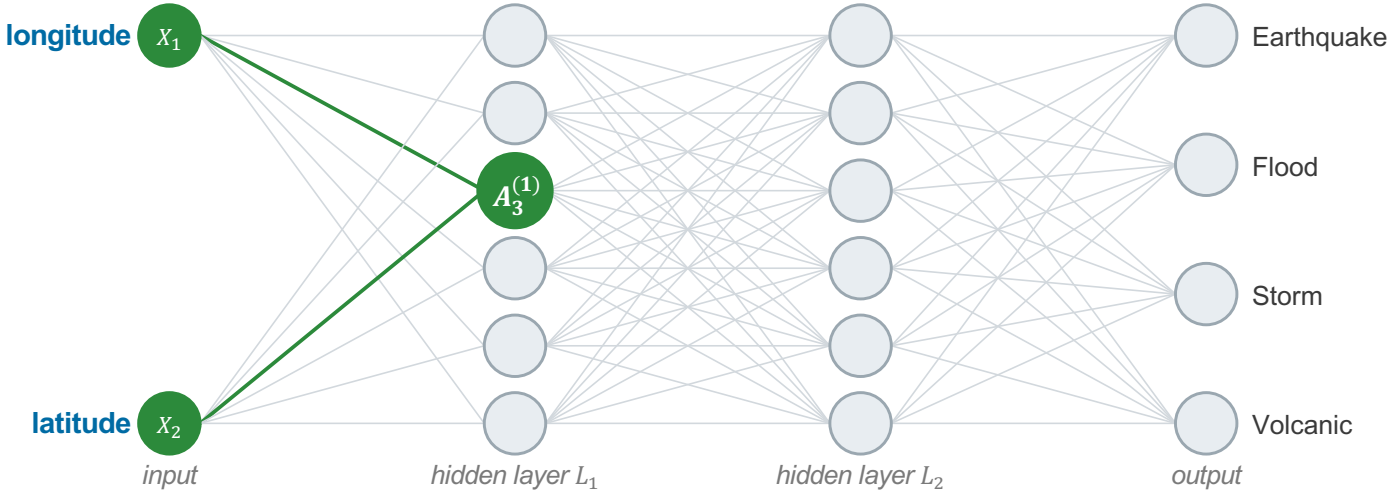


$\boldsymbol{w_{kj}^{(l)}}$ is the weight that connects neuron $\boldsymbol{j}$ of layer $\boldsymbol{l-1}$ to neuron $\boldsymbol{k}$ of layer $\boldsymbol{l}$.

The activation $\boldsymbol{A_k^{(l)}}$ is the result of a hidden unit $\boldsymbol{h_k^{(l)}}$ processing its input, e.g. $\boldsymbol{A_k^{(1)} = h_k^{(1)}(X)}$.

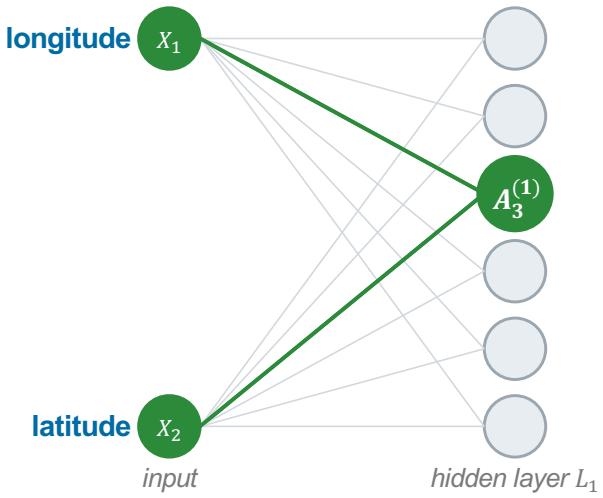
The notation might seem counterintuitive (at least for me it did), but we follow the definition from book p. 404

Each layer treats the activations of the previous layer as inputs.



We get the first layer's activation $A_k^{(1)}$ by passing the input X to the hidden layer $\boldsymbol{h}_{\boldsymbol{k}}^{(1)}$.

A nonlinear activation g makes the network more than a line.



We get the first layer's activation $A_k^{(1)}$ by passing the input X to the hidden layer $\boldsymbol{h_k^{(1)}}$.

A hidden unit $\boldsymbol{h_k^{(l)}}$ is a combination of a linear processing of the previous layer's output:

$$z_k^{(l)} = w_{k0}^{(l)} + \sum_{j=1}^{K_{l-1}} w_{kj}^{(l)} A_j^{(l-1)}$$

and a non-linear transformation function g, such that:

$$A_k^{(l)} = h_k^{(l)}(A^{(l-1)}) = g\left(z_k^{(l)}\right).$$

Without g, $f(X)$ would collapse into a linear model in X.

Activation functions provide non-linear transformations.

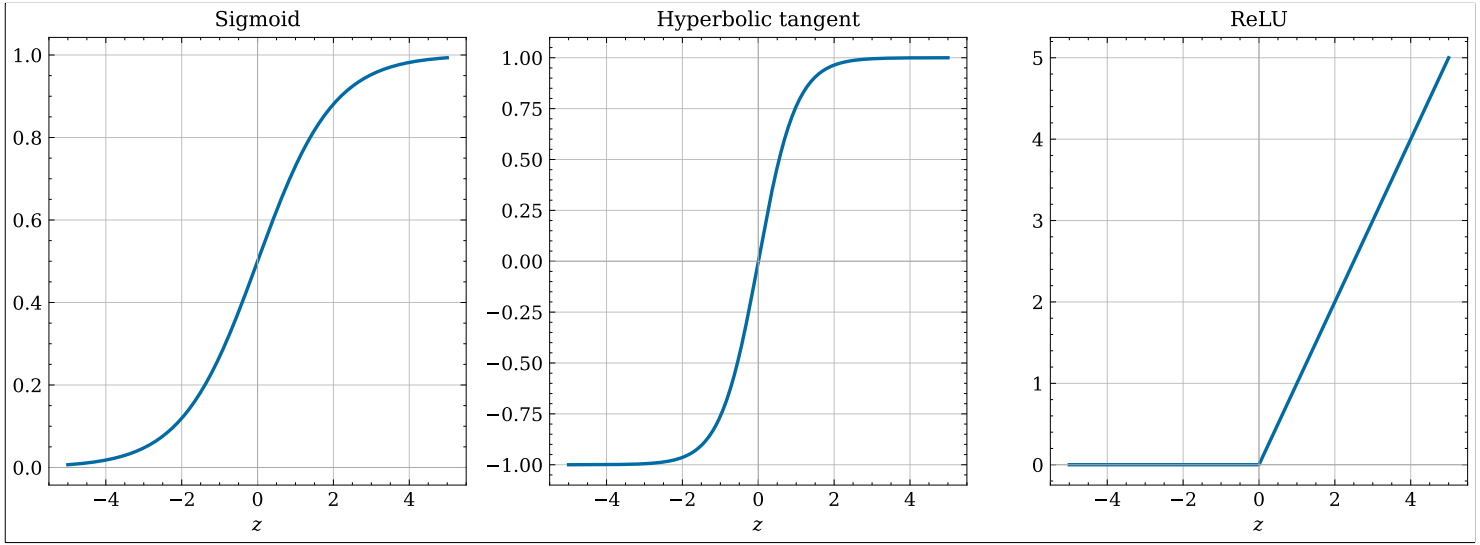


Figure 5.2: activation functions

$$g(z) = \frac{e^z}{1 + e^z}$$

sigmoid

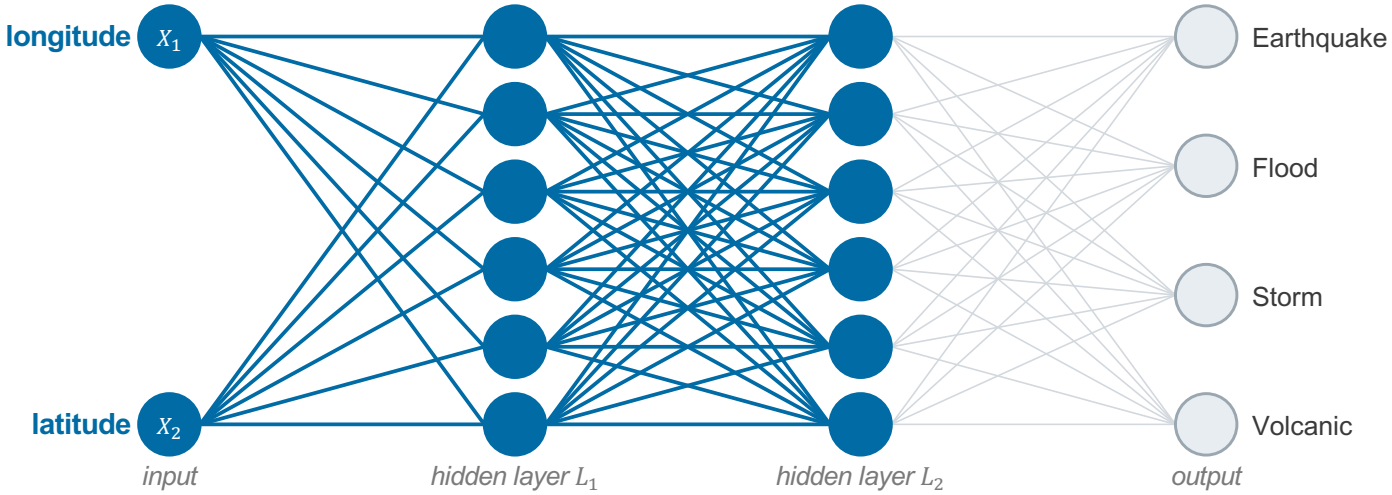
$$g(z) = \tanh(z)$$

tanh

$$g(z) = (z)_+ = \max(0, z)$$

ReLU

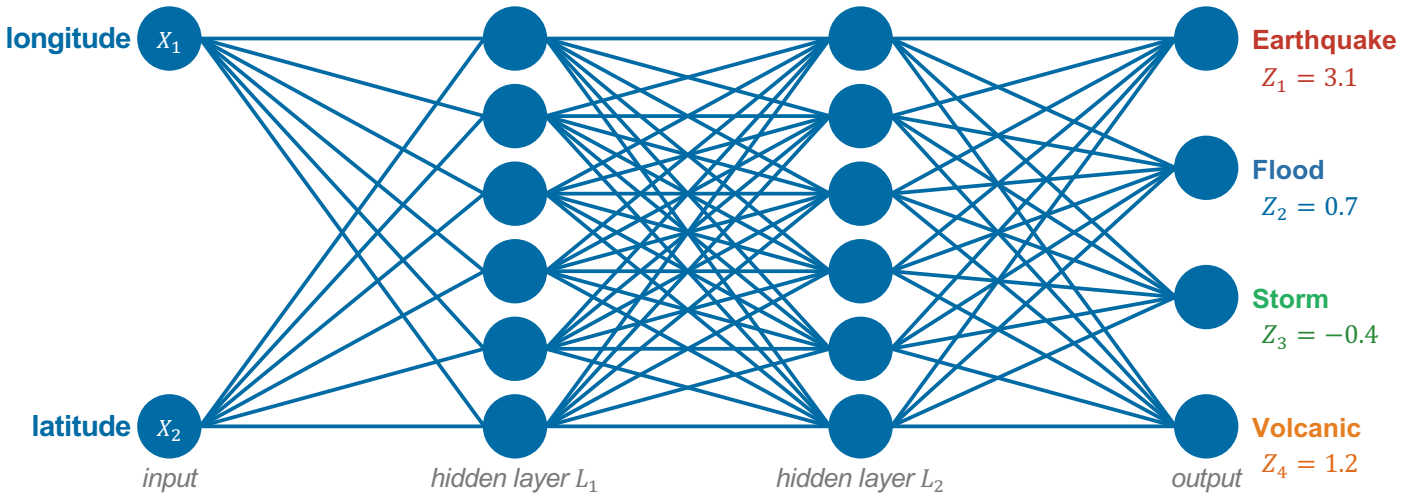
Layer by layer: the multilayer forward pass.



We continue this calculation throughout the network:

$$A_k^{(l)} = h_k^{(l)}(A^{(l-1)}) = g\left(w_{k0}^{(l)} + \sum_{j=1}^{K_{l-1}} w_{kj}^{(l)} A_j^{(l-1)}\right)$$

The output layer computes one linear score Z_m per class.



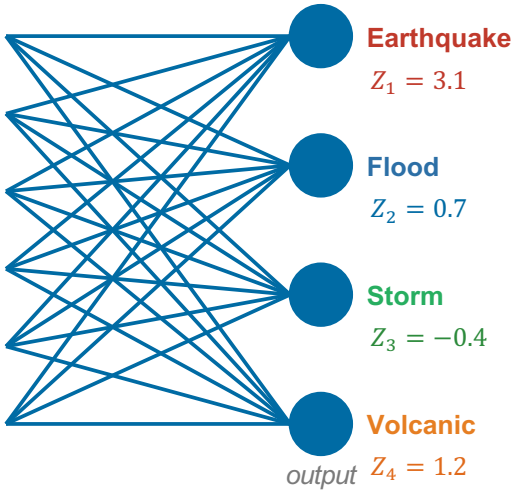
We continue this calculation throughout the network:

$A_k^{(l)} = h_k^{(l)}(A^{(l-1)}) = g\left(w_{k0}^{(l)} + \sum_{j=1}^{K_{l-1}} w_{kj}^{(l)} A_j^{(l-1)}\right)$

… until we get to the final layer.

Where we stop before applying the nonlinear transformation g. This output is referred to as ***logits*** Z_m. We want the final output to represent class probabilities.

Softmax turns the logits Z_m into class probabilities $f_m(X)$.



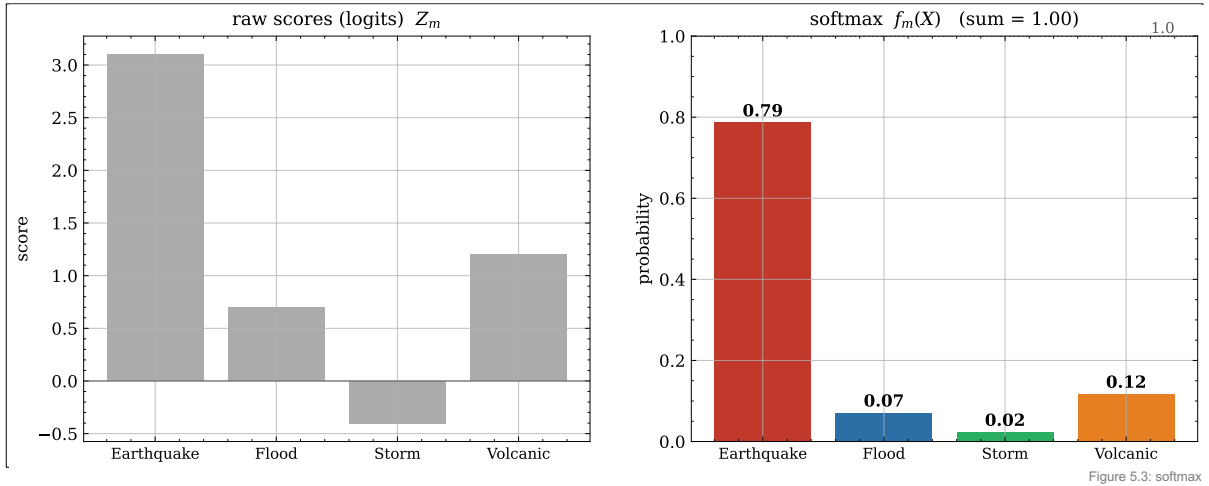


Figure 5.3: softmax

Where we stop before applying the nonlinear transformation g. This output is referred to as ***logits*** Z_m. We want the final output to represent class probabilities.

Just like in multinomial logistic regression we use softmax activation function:

$$\text{softmax}(Z_m) = \frac{e^{Z_m}}{\sum_l e^{Z_l}}$$

Trained end-to-end, the network separates disaster types.

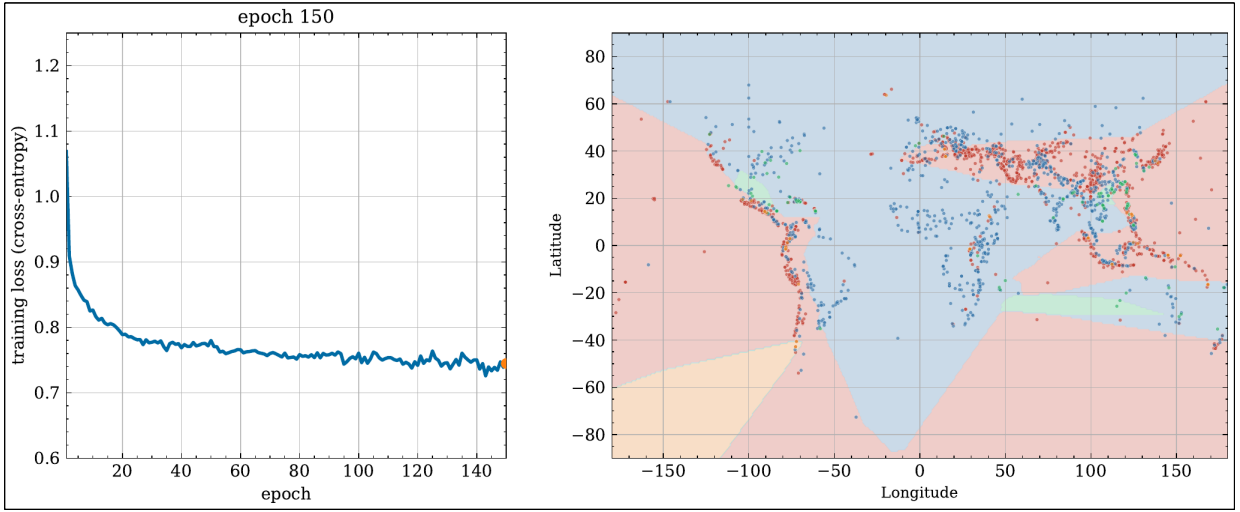
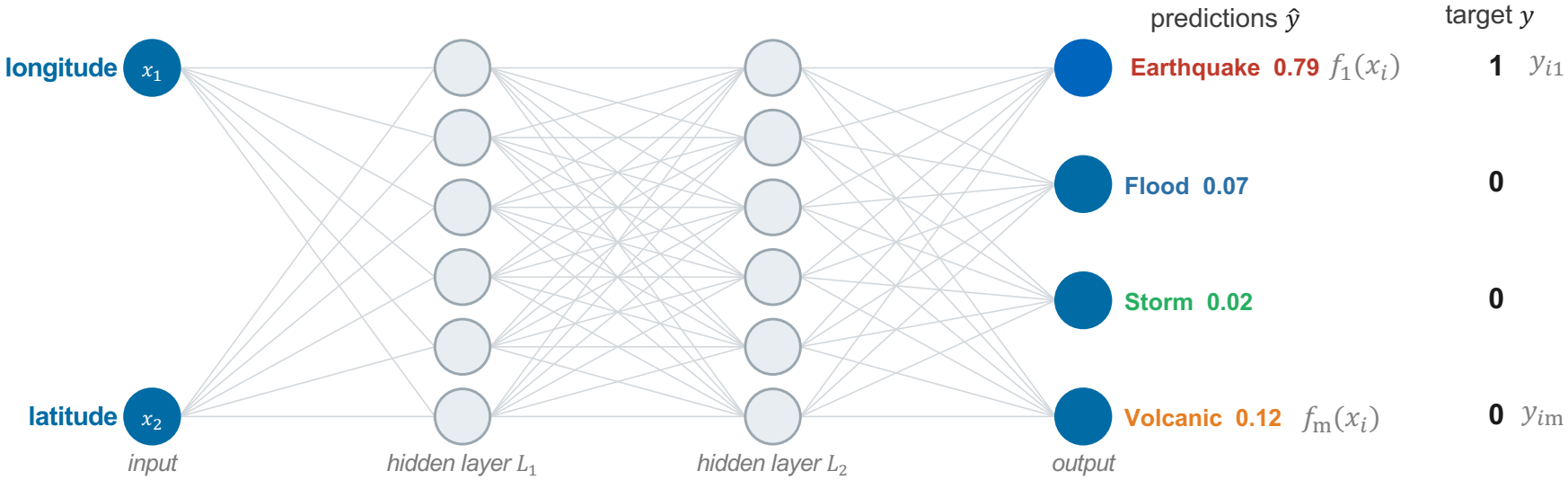


Figure 5.4: training animation

Agenda

- Motivation for neural networks
- Network architecture
- Loss and backpropagation
- Optimization

The loss compares the predicted vector with the target.



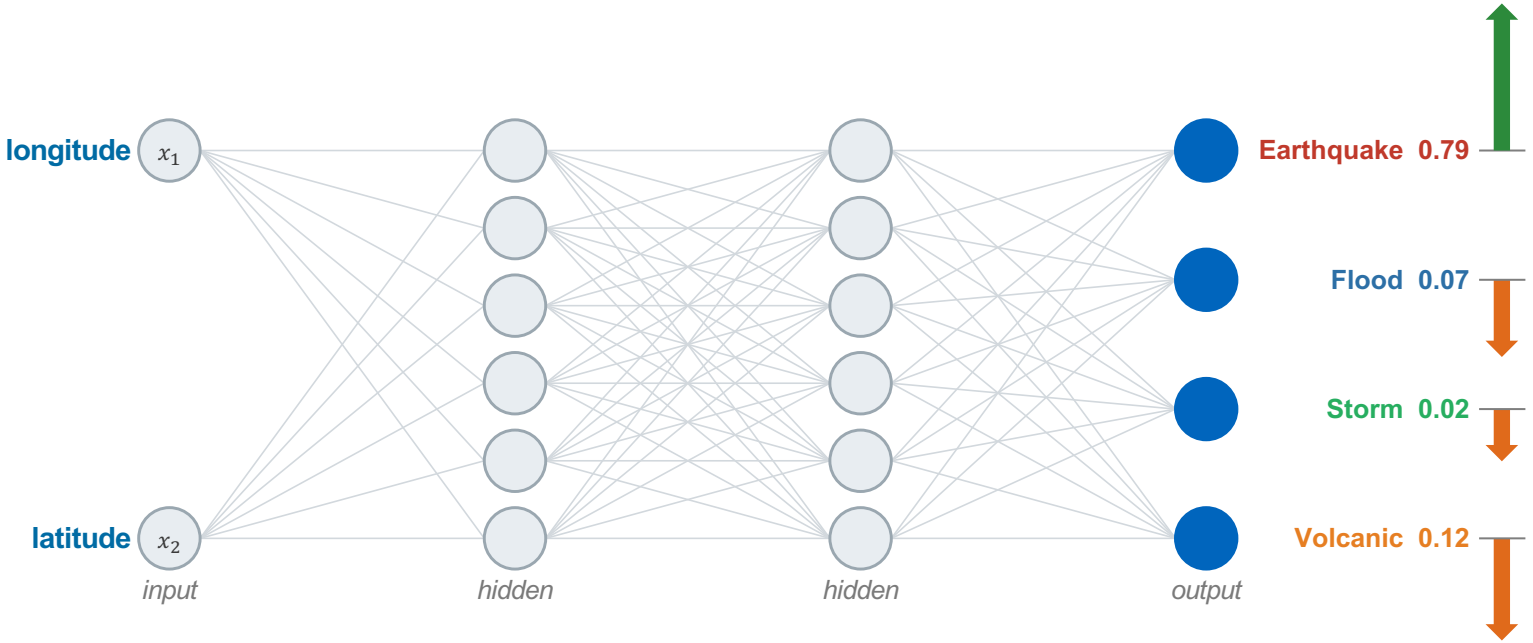
We put all weights $\boldsymbol{w_{kj}^{(l)}}$ into a vector $\boldsymbol{\theta}$ and define our objective (**cross-entropy**) to be:

$$\min_{\theta} R(\theta) = -\sum_{i=1}^{n}\sum_{m=1}^{M} y_{im}\ log(f_m(x_i))$$

For a quantitative response we use the **squared-error loss**:

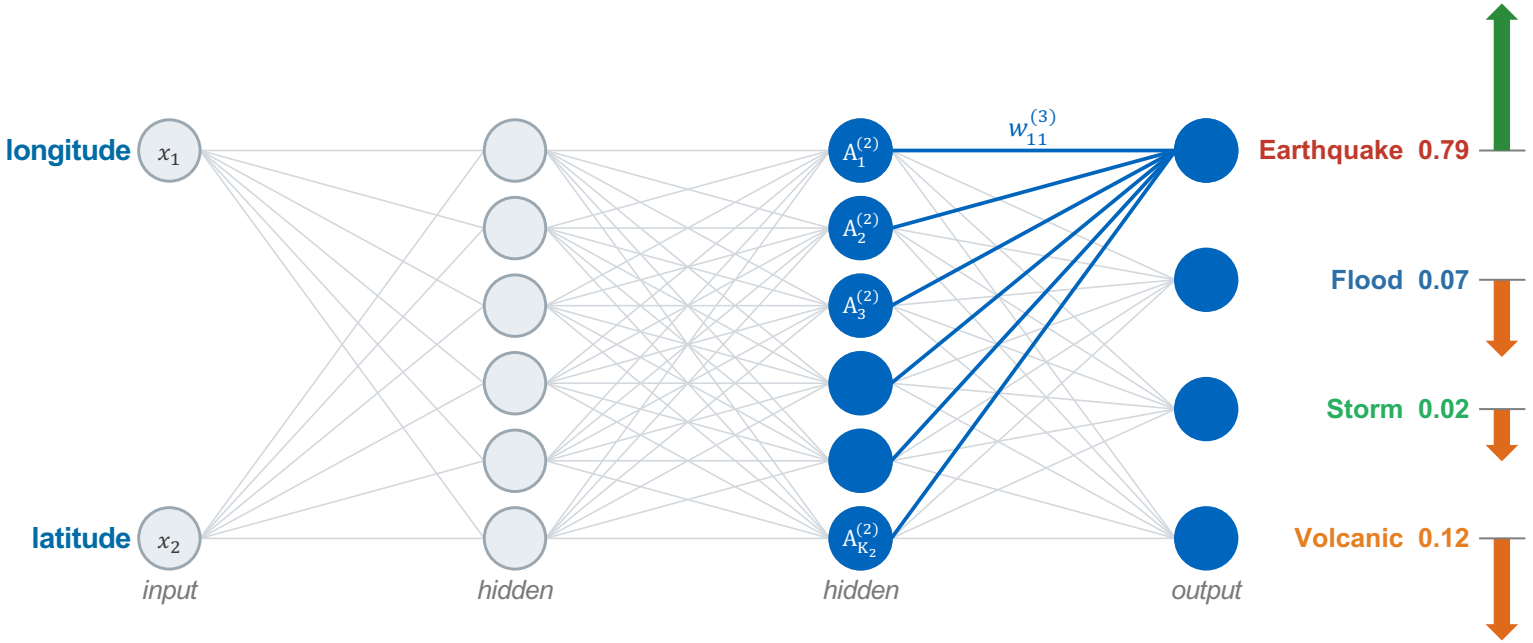
$$\boldsymbol{R(\theta) = \sum_{i=1}^{n} (y_i - f_\theta(x_i))^2}$$

We can derive which output should be larger or smaller.



For this example the target is Earthquake. We want $\hat{y}_1$ **larger** and the other three **smaller**.
Arrow size = how far each output was off.

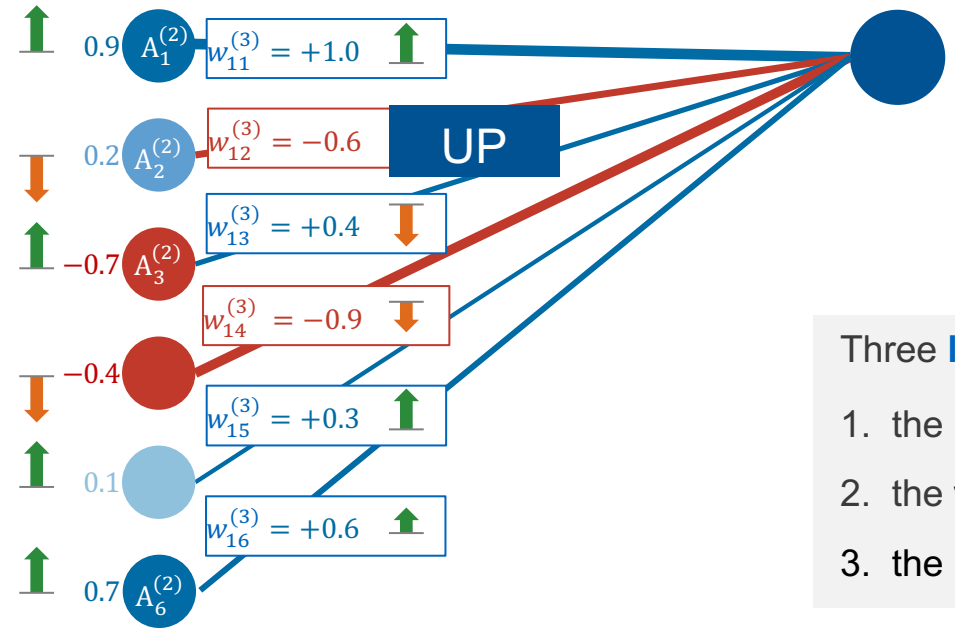
Each parameter contributed differently to the current prediction.



Every parameter in $\boldsymbol{\theta}$ has a direction that reduces this example's error $\boldsymbol{R_i(\theta)}$.

Zoom in on the neuron and its incoming weights.

Every parameter in $\boldsymbol{\theta}$ has a direction that reduces this example's error $\boldsymbol{R_i(\theta)}$.



blue = positive
red = negative
***thickness** = size of weights*

Earthquake z_{Eq}

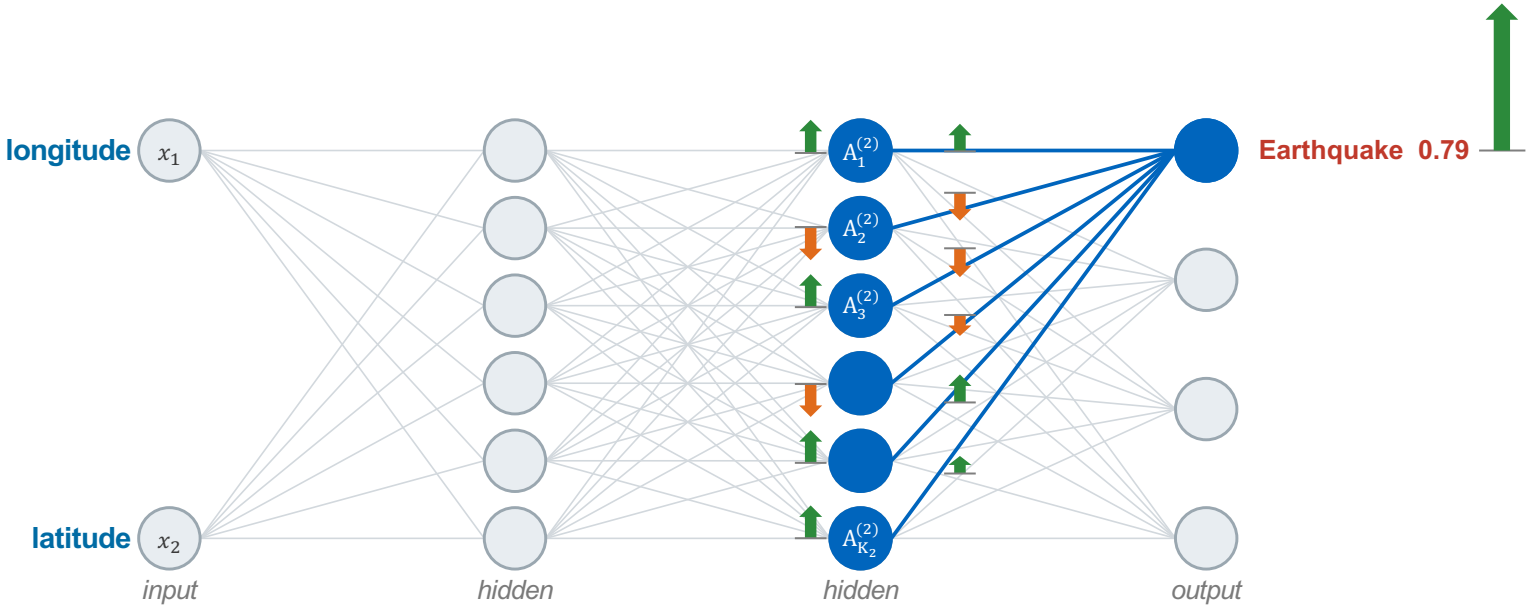
$$= w_{10}^{(3)} + w_{11}^{(3)} A_1^{(2)} + w_{12}^{(3)} A_2^{(2)} + \cdots + w_{16}^{(3)} A_6^{(2)}$$

Three **levers** can change this score:

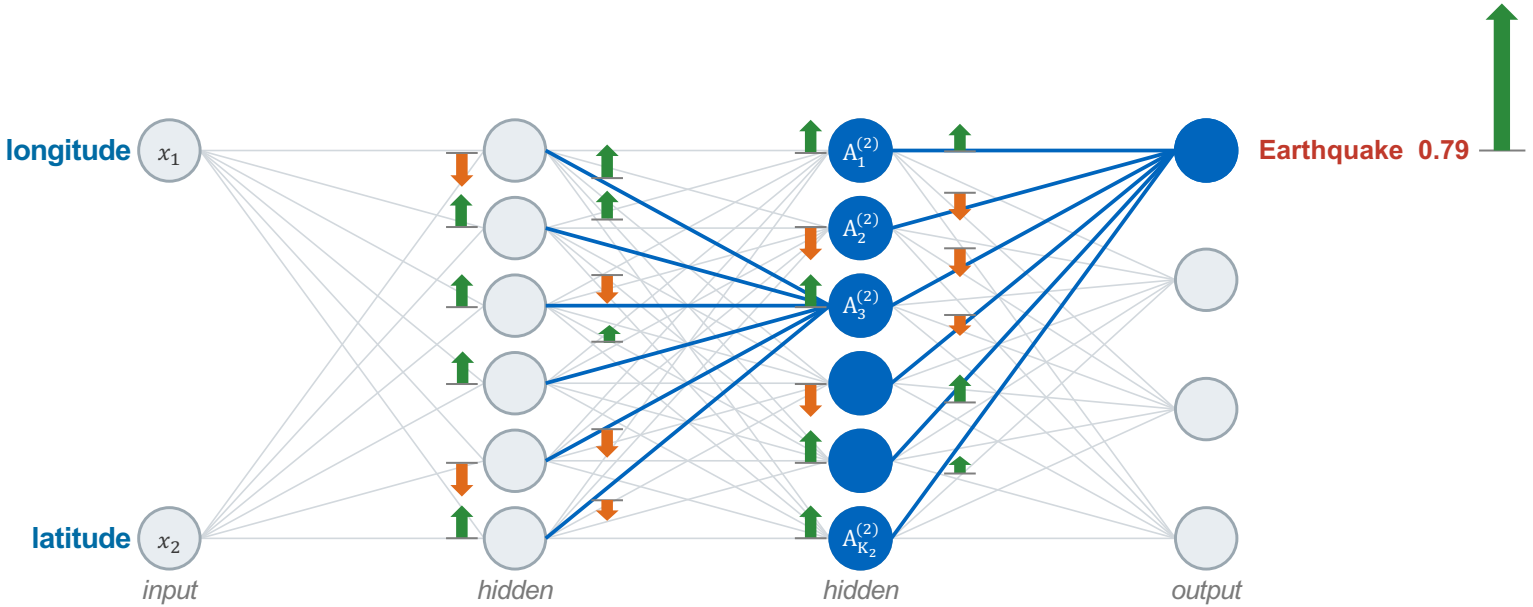
1. the **bias** $w_{10}^{(3)}$
2. the **weights** $w_{1k}^{(3)}$
3. the incoming **activations** $A_k^{(2)}$

Which in turn only depends on the previous layers …

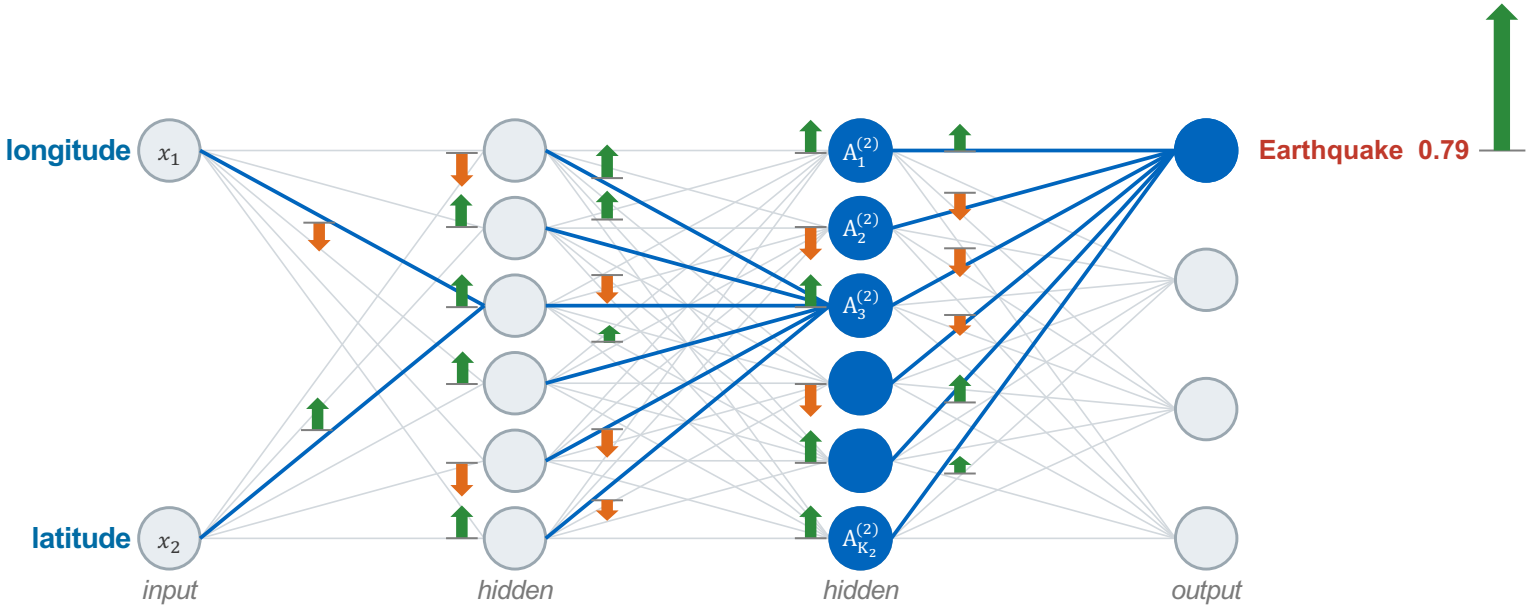
To learn, we nudge the weights toward the target.



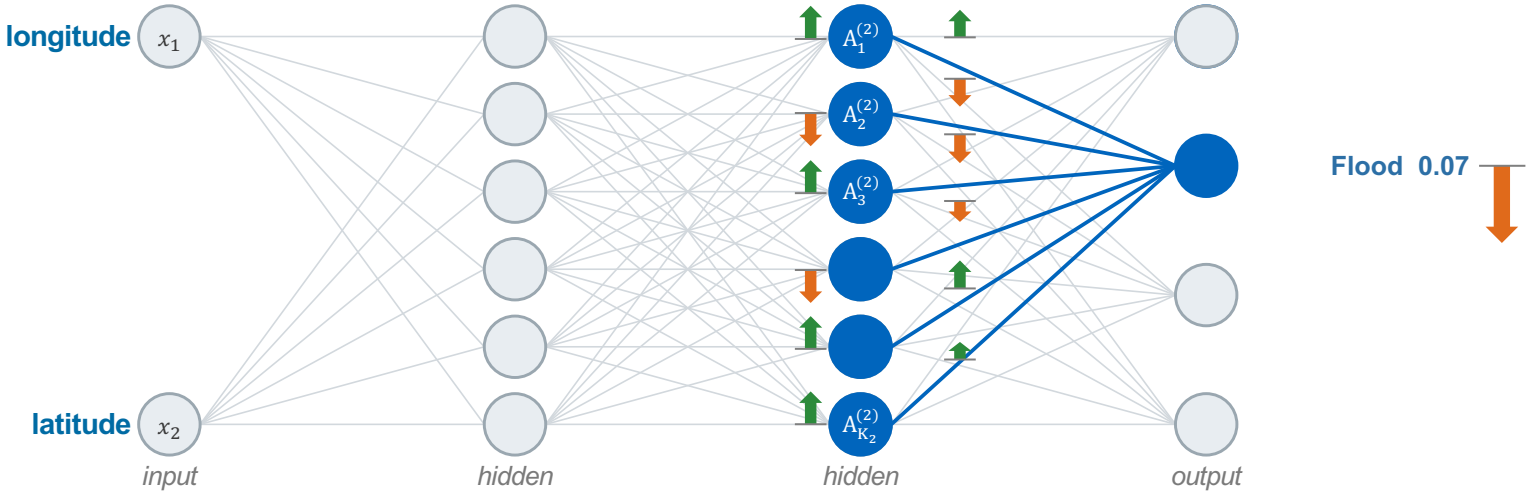
We can backpropagate the changes we would like to see.



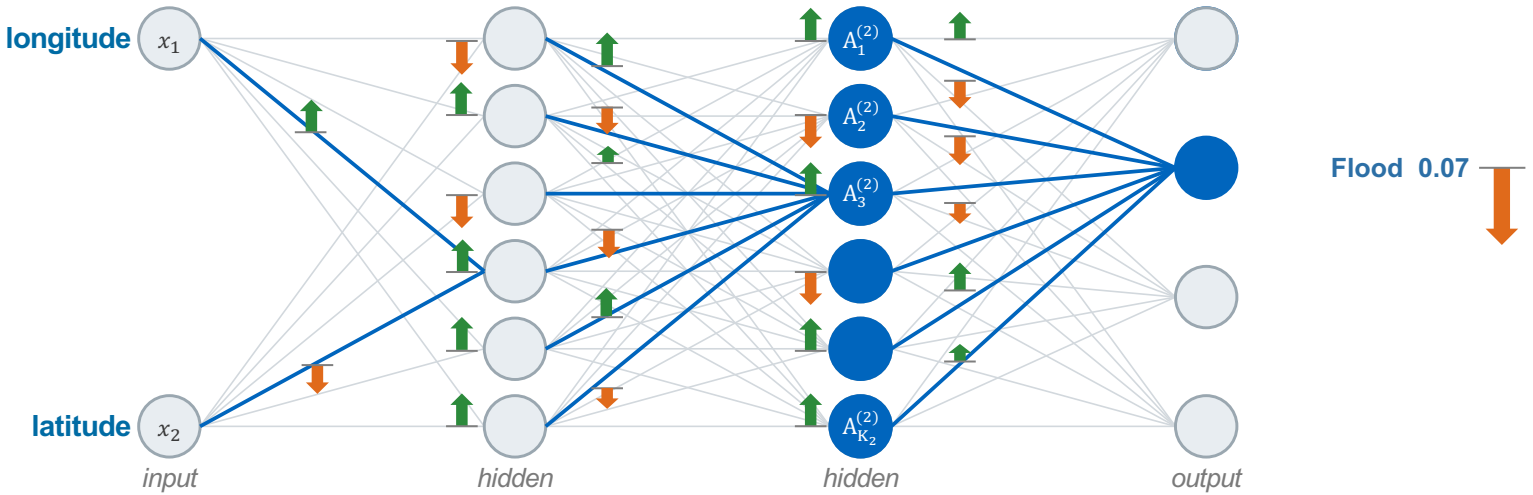
This backpropagation goes all the way to the first weights in our network.



For each output neuron we get different nudges.



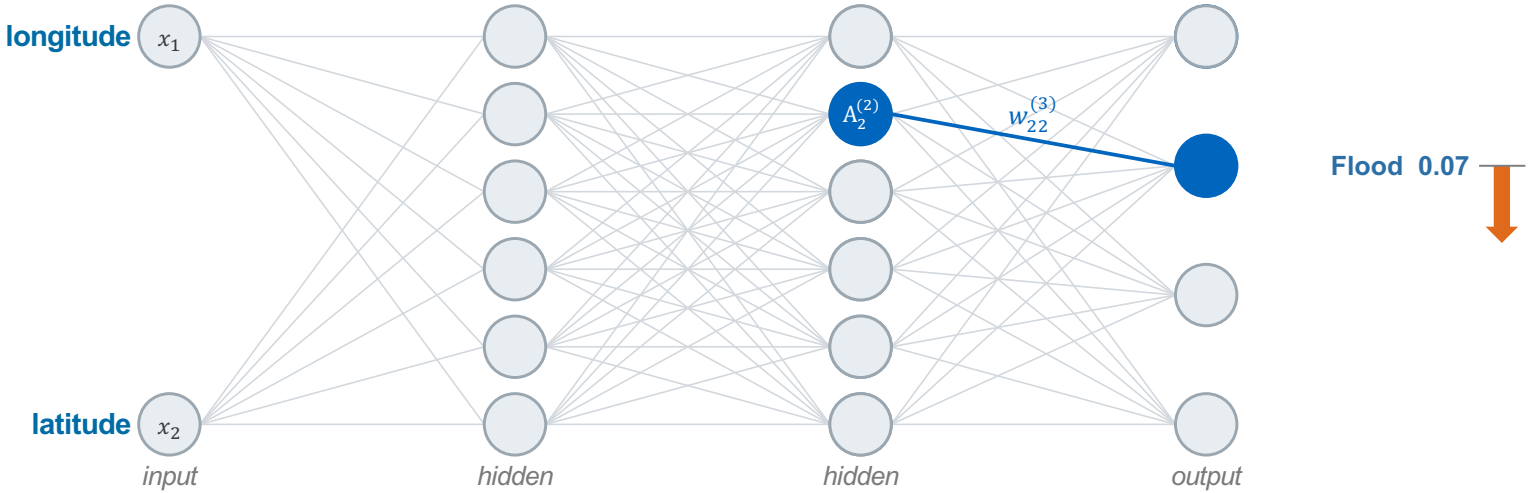
And each of these nudges propagates backwards.



?

How do we tell a machine how every parameter in $\boldsymbol{\theta}$ should change?

We look at a single weight to understand how we adapt it.



How do we tell a machine how every parameter in $\boldsymbol{\theta}$ should change?

We want to get to the minimum loss.

At the current weight θ^t we read the **tangent**, the gradient of the loss.

$$\nabla R_i(\theta^t)$$

We step downhill by the **learning rate ρ**, then move to the next data point:

$$\theta^{t+1} \leftarrow \theta^t - \rho \nabla R_i(\theta^t)$$

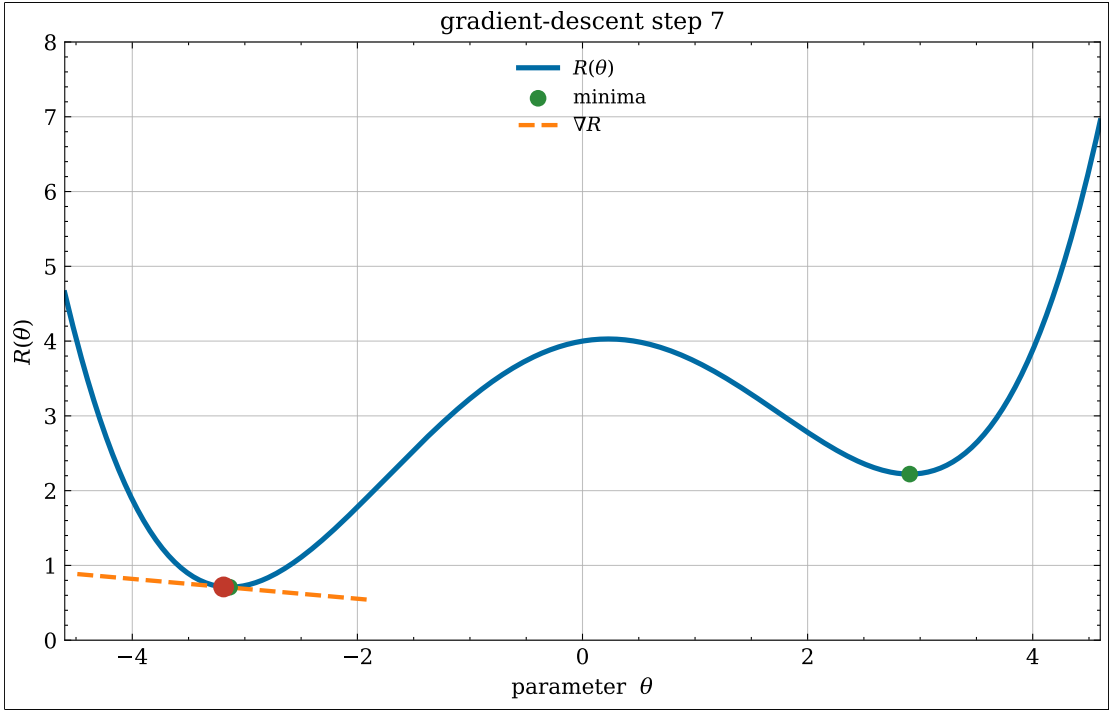
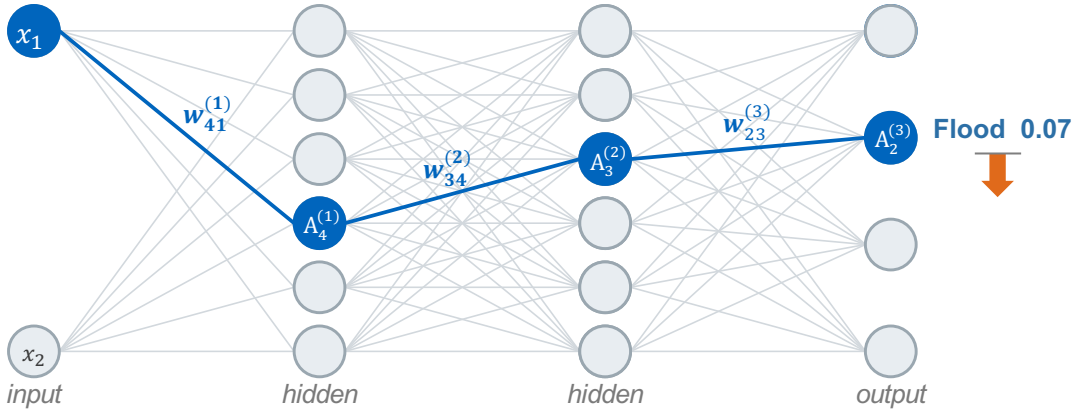


Figure 5.5: gradient descent

We follow this gradient across all weights of our network.

At the current weight θ^t we read the **tangent**, the gradient of the loss.

$$\nabla R_i(\theta^t) = \begin{bmatrix} \vdots \\ \nabla \boldsymbol{w}^{(1)} \\ \vdots \\ \nabla \boldsymbol{w}^{(2)} \\ \vdots \\ \nabla \boldsymbol{w}^{(3)} \\ \vdots \end{bmatrix}$$



We step downhill by **the learning rate ρ**, then move to the next data point:

$$\theta^{t+1} \leftarrow \theta^t - \rho \nabla R_i(\theta^t)$$

?

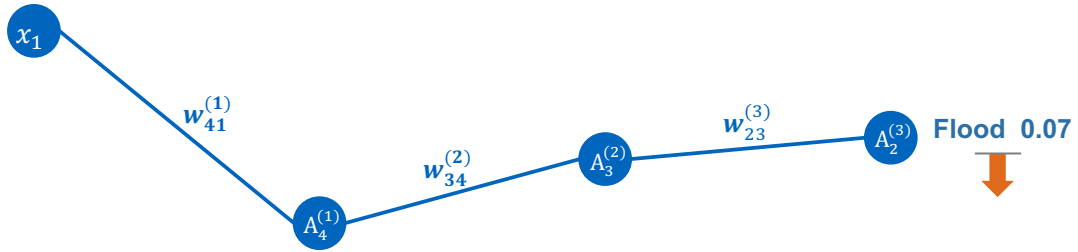
How do we tell a machine how every parameter in $\boldsymbol{\theta}$ should change?

We observe the gradient along a simple network.

At the current weight θ^t we read the **tangent**, the gradient of the loss.

$$\nabla R_i(\theta^t)$$

For simplicity, let's assume this to be our network:



By this assumption our cross-entropy loss boils down to:

$$\boldsymbol{R_i(\theta^t) = -\left[y_i \, log \, A^{(L)} + (1 - y_i) \, log\left(1 - A^{(L)}\right)\right]}$$

And identify the impact of each parameter.

At the current weight θ^t we read the **tangent**, the gradient of the loss.

$$\nabla R_i(\theta^t) = \frac{\partial R_i(\theta)}{\partial \theta}\Big|_{\theta=\theta^t}$$

Gradient is just the partial derivative in each dimension of our parameters.

Start by asking:

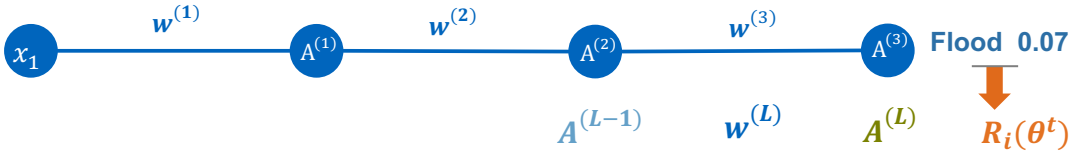
$$\frac{\partial R_i(\theta^t)}{\partial w^{(L)}} = \frac{\partial R_i(\theta^t)}{\partial A^{(L)}} \frac{\partial A^{(L)}}{\partial z^{(L)}} \frac{\partial z^{(L)}}{\partial w^{(L)}}$$

… that's what we call the **chain rule**

Instead of asking how $w^{(L)}$ impacts $R_i(\theta^t)$.

We ask much simpler questions.

For simplicity, let's assume this to be our network:



Let's walk our way backwards:

$$R_i(\theta^t) = -\left[y_i \log\left(A^{(L)}\right) + (1 - y_i) \log\left(1 - A^{(L)}\right)\right]$$

$$A^{(L)} = g\left(z^{(L)}\right)$$

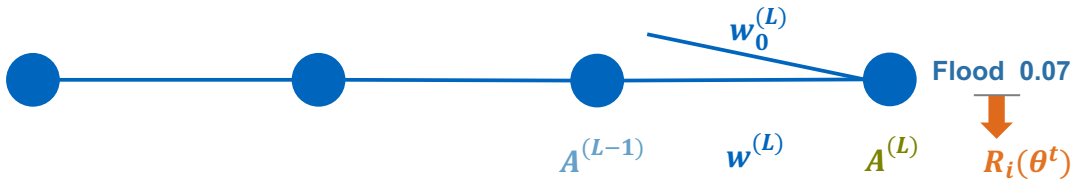
$$z^{(L)} = w^{(L)} A^{(L-1)} + w_0^{(L)}$$

The chain rule finds these gradients efficiently.

$$\frac{\partial R_i(\theta^t)}{\partial w^{(L)}} = \frac{\partial R_i(\theta^t)}{\partial A^{(L)}} \cdot \frac{\partial A^{(L)}}{\partial z^{(L)}} \cdot \frac{\partial z^{(L)}}{\partial w^{(L)}}$$

… that's what we call the **chain rule**

For simplicity, let's assume this to be our network:



So, what is the value of each term?

$$\frac{\partial R_i(\theta^t)}{\partial A^{(L)}} = -\frac{y_i}{A^{(L)}} + \frac{1-y_i}{1-A^{(L)}}$$

$$\frac{\partial A^{(L)}}{\partial z^{(L)}} = g'\left(z^{(L)}\right)$$

$$\frac{\partial z^{(L)}}{\partial w^{(L)}} = A^{(L-1)}$$

$$R_i(\theta^t) = -\left[y_i \log(A^{(L)}) + (1-y_i)\log\left(1-A^{(L)}\right)\right]$$

$$A^{(L)} = g\left(z^{(L)}\right)$$

$$z^{(L)} = w^{(L)} A^{(L-1)} + w_0^{(L)}$$

But what about all the other parameters?

$$\nabla R(\theta^t) = \frac{\partial R(\theta)}{\partial \theta}\Big|_{\theta=\theta^t}$$

The chain rule finds these gradients efficiently.

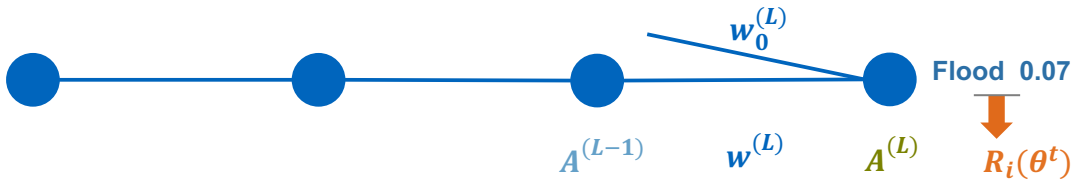
$$\frac{\partial R_i(\theta^t)}{\partial w^{(L)}} = \frac{\partial R_i(\theta^t)}{\partial A^{(L)}} \cdot \frac{\partial A^{(L)}}{\partial z^{(L)}} \cdot \frac{\partial z^{(L)}}{\partial w^{(L)}}$$

? So, what changes for $w_0^{(L)}$?

$$\frac{\partial R_i(\theta^t)}{\partial w_0^{(L)}} = \frac{\partial R_i(\theta^t)}{\partial A^{(L)}} \cdot \frac{\partial A^{(L)}}{\partial z^{(L)}} \cdot \frac{\partial z^{(L)}}{\partial w_0^{(L)}}$$

$$\frac{\partial z^{(L)}}{\partial w_0^{(L)}} = 1$$

For simplicity, let's assume this to be our network:



? What changes for $A^{(L-1)}$?

$$\frac{\partial R_i(\theta^t)}{\partial A^{(L-1)}} = \frac{\partial R_i(\theta^t)}{\partial A^{(L)}} \cdot \frac{\partial A^{(L)}}{\partial z^{(L)}} \cdot \frac{\partial z^{(L)}}{\partial A^{(L-1)}}$$

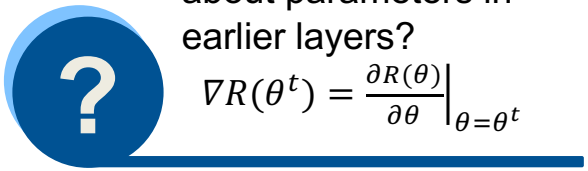
$$\frac{\partial z^{(L)}}{\partial A^{(L-1)}} = w^{(L)}$$

The chain rule finds these gradients efficiently.

$$\frac{\partial R_i(\theta^t)}{\partial w^{(L)}} = \frac{\partial R_i(\theta^t)}{\partial A^{(L)}} \cdot \frac{\partial A^{(L)}}{\partial z^{(L)}} \cdot \frac{\partial z^{(L)}}{\partial w^{(L)}}$$

$$\frac{\partial R_i(\theta^t)}{\partial A^{(L-1)}}$$

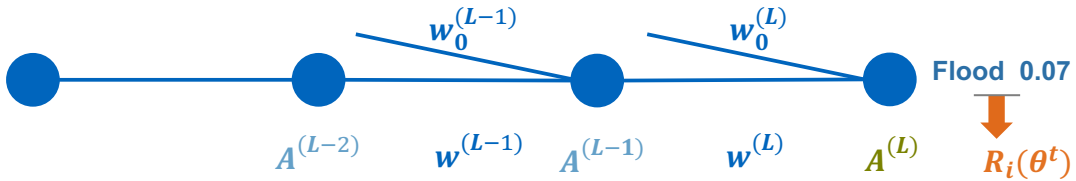
$$\frac{\partial R_i(\theta^t)}{\partial w_0^{(L)}}$$



Lucky shot! How about parameters in earlier layers?

$$\nabla R(\theta^t) = \frac{\partial R(\theta)}{\partial \theta}\Big|_{\theta=\theta^t}$$

For simplicity, let's assume this to be our network:



$$\frac{\partial R_i(\theta^t)}{\partial w^{(L-1)}} = \frac{\partial R_i(\theta^t)}{\partial A^{(L)}} \cdot \frac{\partial A^{(L)}}{\partial z^{(L)}} \cdot \frac{\partial z^{(L)}}{\partial A^{(L-1)}} \cdot \frac{\partial A^{(L-1)}}{\partial z^{(L-1)}} \cdot \frac{\partial z^{(L-1)}}{\partial w^{(L-1)}}$$

$$\frac{\partial R_i(\theta^t)}{\partial A^{(L-2)}} \quad \dots$$

$$\frac{\partial R_i(\theta^t)}{\partial w_0^{(L-1)}}$$

Backprop uses the activation derivative $g'(z)$.

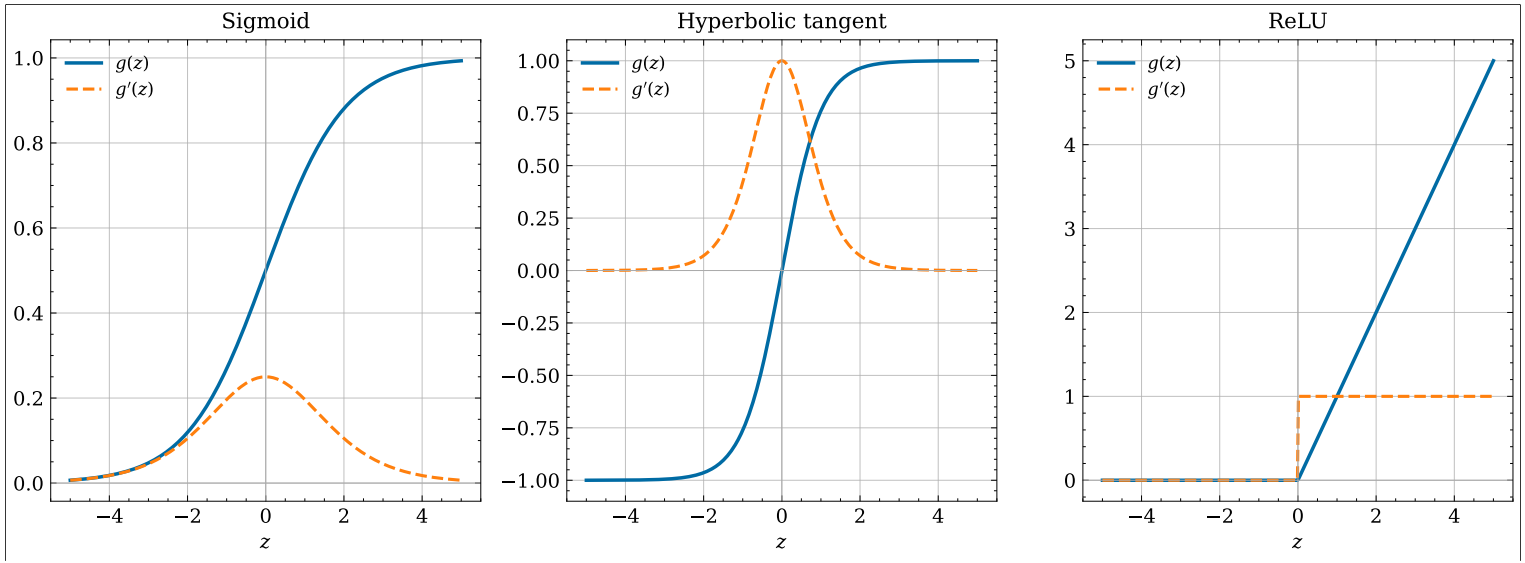


Figure 5.6: activation derivatives

Loss and backpropagation

Agenda

- Motivation for neural networks
- Network architecture
- Loss and backpropagation
- Optimization

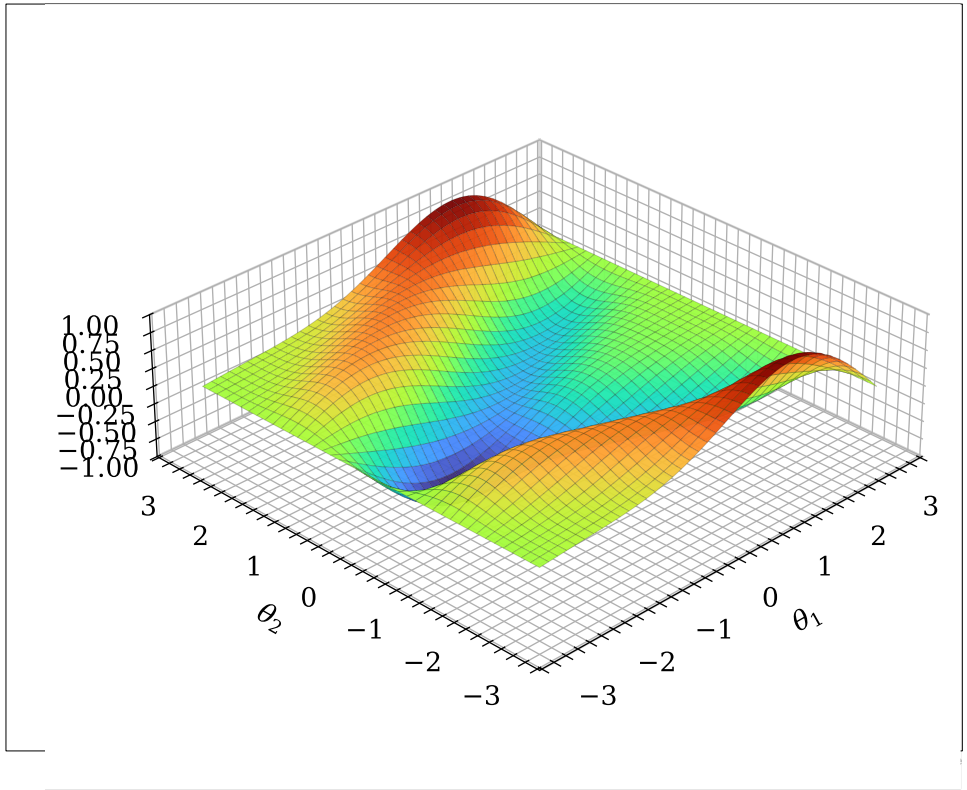
Gradient descent generalizes to higher dimensional spaces.

At the current weight θ^t we read the **tangent**, the gradient of the loss:

$$\nabla R_i(\theta^t)$$

We step downhill by the **learning rate ρ**, then move to the next data point:

$$\theta^{t+1} \leftarrow \theta^t - \rho \nabla R_i(\theta^t)$$



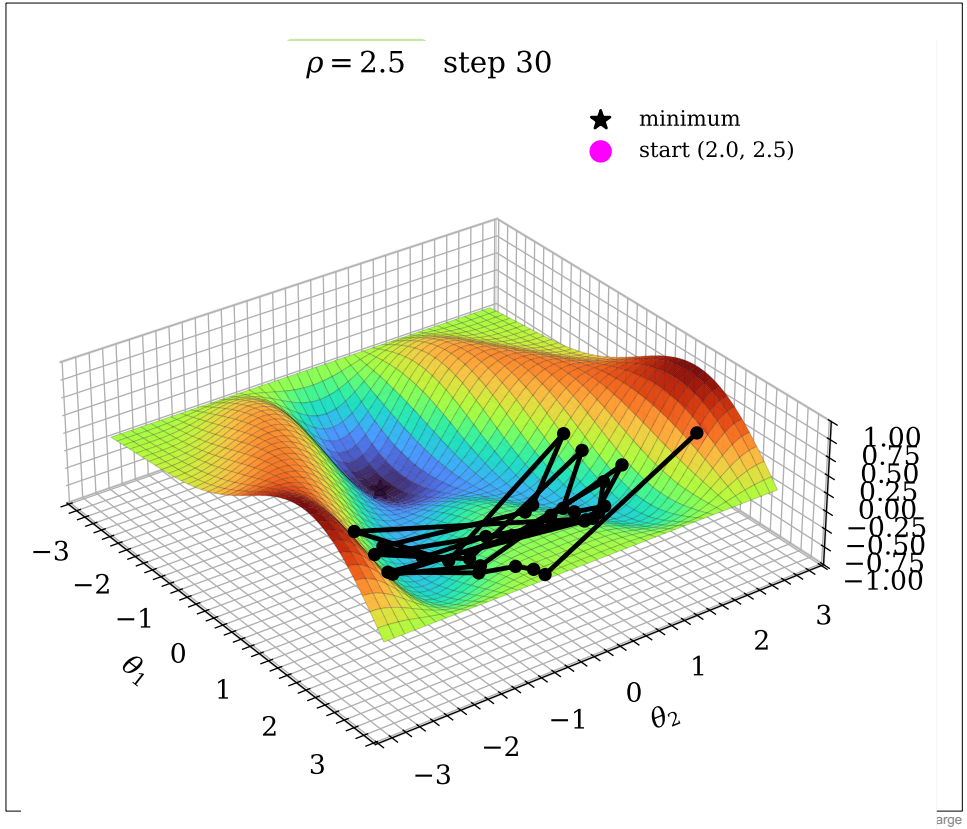
With ρ too big we overshoot and never find the minimum.

At the current weight θ^t we read the **tangent**, the gradient of the loss:

$$\nabla R_i(\theta^t)$$

We step downhill by the **learning rate ρ**, then move to the next data point:

$$\theta^{t+1} \leftarrow \theta^t - \rho \nabla R_i(\theta^t)$$



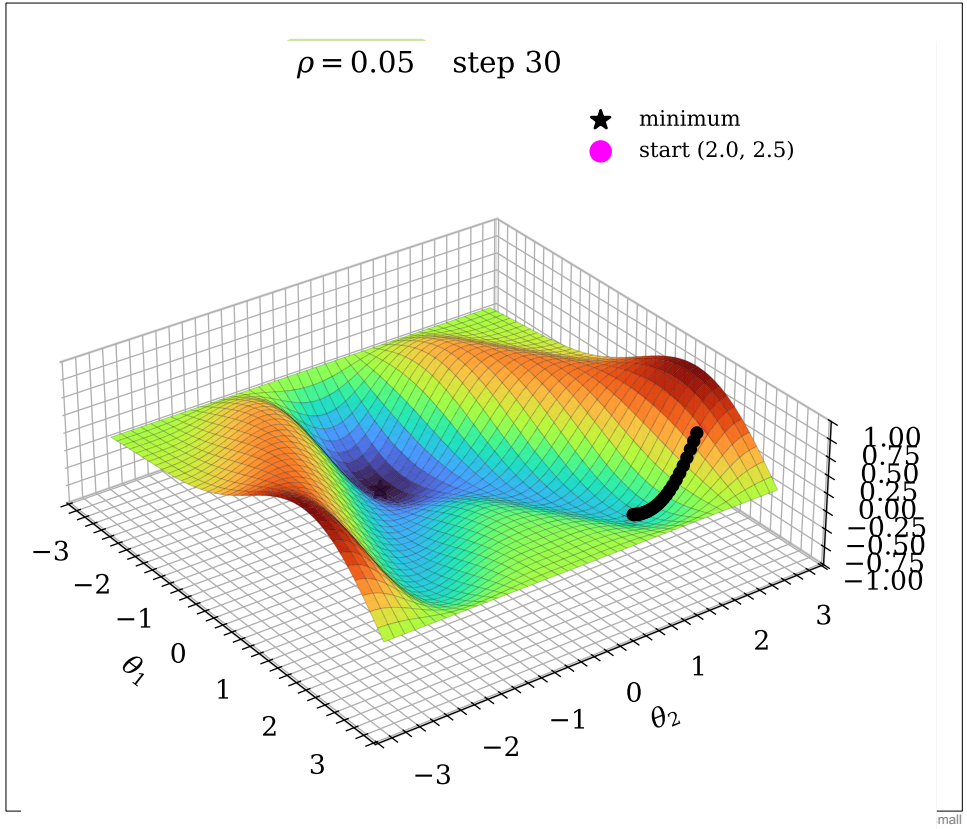
With ρ too small we get stuck in a loss valley.

At the current weight θ^t we read the **tangent**, the gradient of the loss:

$$\nabla R_i(\theta^t)$$

We step downhill by the **learning rate ρ**, then move to the next data point:

$$\theta^{t+1} \leftarrow \theta^t - \rho \nabla R_i(\theta^t)$$



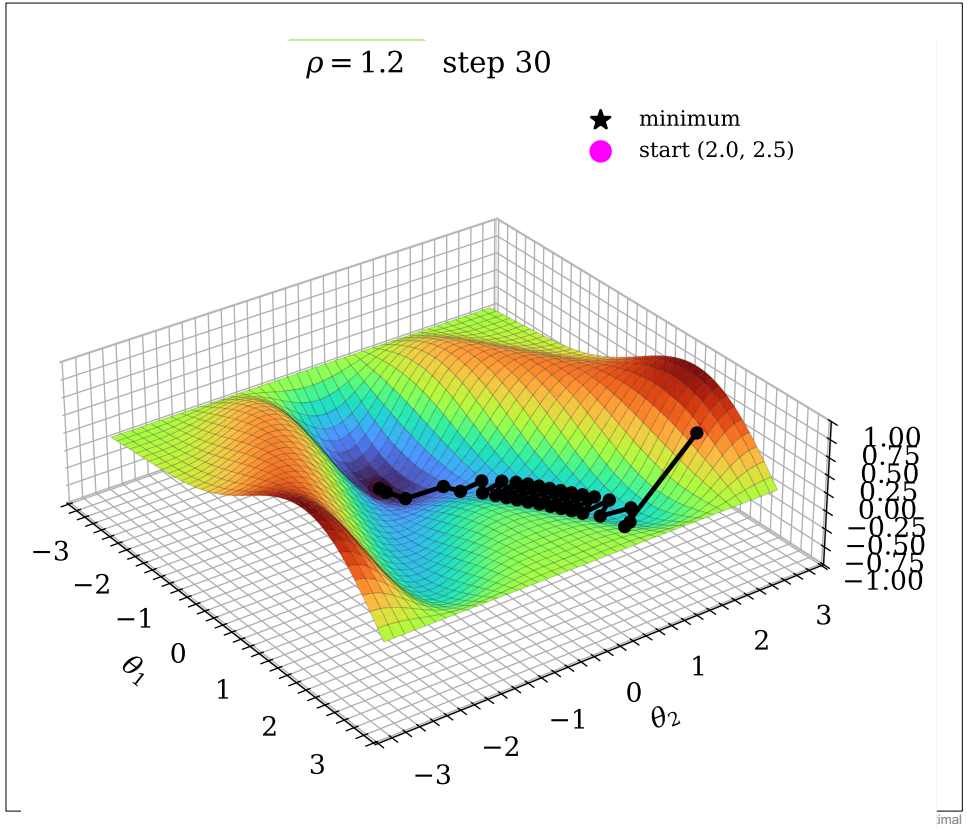
The right learning rate brings us to the minimum.

At the current weight θ^t we read the **tangent**, the gradient of the loss:

$$\nabla R_i(\theta^t)$$

We step downhill by the **learning rate ρ**, then move to the next data point:

$$\theta^{t+1} \leftarrow \theta^t - \rho \nabla R_i(\theta^t)$$



One sample gives a noisy gradient, averaging approaches the true one.

At the current weight θ^t we read the **tangent**, the gradient of the loss:

$$\nabla R_i(\theta^t)$$

We do so for a **mini-batch B** of m samples of our data $\{(x_i, y_i)\}_{i=1}^N$. We step downhill by the learning rate $\boldsymbol{\rho}$ in the direction of the average gradient, then move to the next mini-batch:

$$\theta^{(t+1)} \leftarrow \theta^t - (\rho/m)\sum_{(i\in B)} \nabla R_i(\theta^t)$$

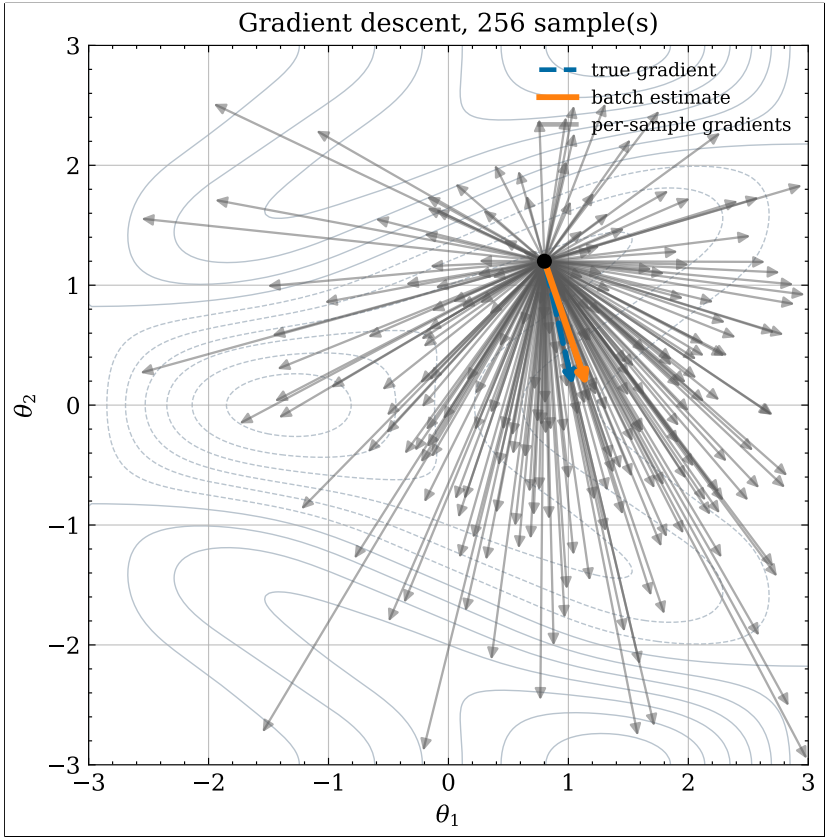


Figure 5.11: sgd mini-batch

Momentum keeps a running direction.

At the current weight θ^t we read the **tangent**, the gradient of the loss:

$$\nabla R_i(\theta^t)$$

We fold it into a **running velocity v** that keeps a fraction γ of the previous direction, then move along v:

$$v \leftarrow \gamma v - \rho \nabla R_i(\theta^t)$$

$$\theta^{t+1} \leftarrow \theta^t + v$$